



AI-Powered Analytics Dashboard

Work Breakdown Structure

Date: March 10, 2026

Version: 1.0

Mode: Scope (no estimates)

Prepared by: ZenLabs

Client: Confidential

Executive Summary

This document defines the Work Breakdown Structure for an AI-powered analytics dashboard proof of concept. The system enables 5-10 internal users to query approximately 50 sales and inventory spreadsheets using natural language, receiving instant visual answers via a web dashboard. The recommended approach is a lightweight custom build using Next.js, Python FastAPI, PostgreSQL, and the OpenAI API, deployed on managed cloud services. The project is scoped as a CEO demo/POC within a \$15K budget constraint. Key risks include LLM query accuracy on financial data and spreadsheet format inconsistency across the 50-file dataset. The architecture prioritizes demo impact and future extensibility over production hardening.

Technical Feasibility

Each major requirement was assessed for technical feasibility. The overall project is viable with known technology, though the NL query accuracy requires careful prompt engineering and validation.

Requirement	Feasibility	Notes
Natural language query over structured data	Viable	OpenAI GPT-4 with structured prompting can generate SQL from natural language. Accuracy depends on schema clarity and prompt engineering. Technical spike recommended.
Spreadsheet data ingestion (50 files)	Viable	Python pandas/openpyxl can parse Excel/CSV. Main risk is format inconsistency across files. Requires a normalization layer.
Real-time query responses	Viable	PostgreSQL queries on normalized data return sub-second. LLM API latency adds 1-3 seconds. Acceptable for demo.
Dashboard with charts and KPIs	Viable	Standard web development with mature charting libraries (Recharts, Chart.js). No technical risk.
5-10 concurrent users	Viable	Trivial load. Managed hosting (Vercel + Railway) handles this without special scaling.
\$15K budget constraint	Viable with constraints	Feasible as a POC with AI-augmented development. Not sufficient for production-grade system. Scope must stay disciplined.

Approach Analysis

Three architectural approaches were evaluated against the project constraints. Approach B (Lightweight Custom Build) was selected as the recommended path forward.

Factor	A: No-Code (Retool)	B: Custom Build (Recommended)	C: AI Platform (Dify)
Stack	Retool + OpenAI API + Airtable	Next.js + FastAPI + PostgreSQL + OpenAI	Dify + React + PostgreSQL
Quality	Limited customization, generic look	Full control over UX, polished demo	Good AI pipeline, platform dependency
Speed	1-2 weeks	3-4 weeks (AI-augmented)	2-3 weeks
Budget Fit	Low dev cost, ongoing licensing	Fits \$15K, minimal ongoing cost	\$10-13K dev, Dify dependency
Demo Impact	Medium — looks like a tool	High — looks like a product	High — if frontend is polished
NL Capability	Basic — limited control	Strong — full prompt engineering control	Excellent — Dify handles RAG natively

Factor	A: No-Code (Retool)	B: Custom Build (Recommended)	C: AI Platform (Dify)
Path to Prod	Locked in Retool ecosystem	Real codebase to evolve	Tied to Dify ecosystem

Decision: Approach B selected. Best balance of demo impact, budget fit, NL query control, and future extensibility. AI-augmented development (Claude Code) compresses the timeline to fit within the \$15K budget.

Work Breakdown Structure

The WBS is organized by SDLC phase at module level. Each module represents a deliverable unit of work with defined scope, dependencies, and risk assessment. This is scope mode — no time estimates are included.

Phase 1: Discovery & Requirements

ID	Module	Scope	Key Deliverables	Dependencies	Risk
1.1	Requirements Documentation	Formalize the brief into structured requirements. Define user personas (CEO, analyst, data admin), map user stories for the NL query flow, dashboard views, and data management.	Requirements doc, user story map, acceptance criteria	None	Low
1.2	Data Audit & Schema Definition	Audit all 50 spreadsheets for format consistency, common columns, and data quality issues. Define a normalized target schema for sales and inventory data.	Data audit report, normalized schema, quality assessment	None	Medium
1.3	NL Query Technical Spike	Validate that OpenAI GPT-4 can translate natural language questions into accurate SQL against the defined schema. Test 10-15 representative queries.	Working spike, accuracy benchmarks, prompt patterns	1.2	Medium

Phase 2: System Design & Architecture

ID	Module	Scope	Key Deliverables	Dependencies	Risk
2.1	Architecture Design	Define high-level architecture: Next.js frontend (Vercel), Python FastAPI backend (Railway), PostgreSQL, OpenAI API. Define API contracts and NL query pipeline.	Architecture diagram, API contracts, infra blueprint	1.1, 1.2, 1.3	Low
2.2	Data Model & Pipeline Design	Design the ingestion pipeline (spreadsheet parsing, normalization, PostgreSQL loading), schema migrations, and weekly update workflow.	Data model ERD, pipeline design, ingestion workflow spec	1.2, 2.1	Low
2.3	Security & Auth Design	Design simple authentication (NextAuth email login), basic roles (admin vs viewer), API key management for OpenAI.	Auth flow design, role definitions, key management	2.1	Low

Phase 3: Development

ID	Module	Scope	Key Deliverables	Dependencies	Risk
3.1	Data Ingestion Pipeline	Build Python pipeline to parse Excel/CSV, normalize data, load into PostgreSQL. Include validation, error reporting, support for weekly incremental updates.	Ingestion service, validator, error reporter, upload UI	2.2	Medium

ID	Module	Scope	Key Deliverables	Dependencies	Risk
3.2	NL Query Engine	Build the core NL query engine: user question to SQL generation via OpenAI GPT-4, query execution, result formatting. Include SQL validation, result sanity checks, conversation context.	Query engine, prompt templates, SQL validator, formatter	3.1, 1.3	High
3.3	Dashboard Frontend	Build Next.js dashboard: KPI cards and charts, NL query chat interface, data management view (upload, ingestion status). Responsive, polished for CEO demo.	Dashboard UI, chart components, NL chat interface, upload page	2.1, 3.2	Low
3.4	Authentication Module	Implement NextAuth email login, simple role system (admin: upload data; viewer: query only), session management, protected routes.	Auth implementation, role-based routing, user management	2.3	Low
3.5	API Layer	Build FastAPI backend: NL query endpoint, data upload/ingestion trigger, dashboard data (KPIs, aggregations), user session proxy.	REST API, documented endpoints, Pydantic schemas, error handling	3.1, 3.2, 3.4	Low

Phase 4: Quality Assurance

ID	Module	Scope	Key Deliverables	Dependencies	Risk
4.1	NL Query Accuracy Testing	Build test suite of 30-50 representative NL questions. Verify query engine accuracy (target 85%+). Include edge cases: ambiguous queries, out-of-scope, empty results.	Test suite, accuracy benchmarks, edge case report	3.2	Medium
4.2	Integration & E2E Testing	End-to-end testing: login, upload spreadsheet, ask question, receive answer, view dashboard. Cover happy paths and error scenarios with real data.	E2E test suite (Playwright), integration coverage, bug report	3.1-3.5	Low
4.3	Demo Rehearsal & UAT	Conduct demo rehearsal with stakeholders. Prepare scripted CEO questions. Verify all demo scenarios work smoothly. Fix surfaced issues.	Demo script, rehearsal sign-off, final bug fixes	4.1, 4.2	Low

Phase 5: Deployment & DevOps

ID	Module	Scope	Key Deliverables	Dependencies	Risk
5.1	Infrastructure & CI/CD	Set up Vercel (frontend), Railway/Render (backend), managed PostgreSQL. GitHub Actions CI/CD (lint, test, deploy). Environment configs (dev, staging, prod).	Deployed environments, CI/CD pipeline, env configs	3.3, 3.5	Low
5.2	Data Migration & Seeding	Run initial ingestion of all 50 spreadsheets into production DB. Verify data integrity. Document weekly update workflow.	Seeded production DB, verification report, update workflow doc	3.1, 5.1	Medium

Phase 6: Post-Launch & Maintenance

ID	Module	Scope	Key Deliverables	Dependencies	Risk
6.1	Monitoring & Observability	Set up uptime monitoring, error tracking (Sentry free tier), LLM API cost tracking, basic logging and alerting.	Monitoring dashboard, alert config, cost tracking	5.1	Low
6.2	Knowledge Transfer	Document system architecture, data pipeline, NL prompt engineering approach, weekly update process. Hand off to maintainer.	Technical docs, admin guide, NL tuning guide	All prior	Low
6.3	Phase 2 Roadmap	Define Phase 2 based on CEO demo feedback: production hardening, additional data sources, advanced analytics, mobile support.	Phase 2 scope doc, prioritized backlog	Demo feedback	Low

Dependency Map

The table below shows upstream dependencies for each module. Modules cannot begin until all listed dependencies are complete. The critical path runs through: 1.2 → 1.3 → 2.1 → 2.2 → 3.1 → 3.2 → 3.3/3.5 → 4.1/4.2 → 4.3.

Module	Name	Blocked By	Blocks
1.1	Requirements Documentation	None	2.1
1.2	Data Audit & Schema	None	1.3, 2.1, 2.2
1.3	NL Query Spike	1.2	2.1, 3.2
2.1	Architecture Design	1.1, 1.2, 1.3	2.2, 2.3, 3.3, 3.5
2.2	Data Model & Pipeline	1.2, 2.1	3.1
2.3	Security & Auth Design	2.1	3.4
3.1	Data Ingestion Pipeline	2.2	3.2, 3.5, 5.2
3.2	NL Query Engine	3.1, 1.3	3.3, 3.5, 4.1
3.3	Dashboard Frontend	2.1, 3.2	4.2, 5.1
3.4	Authentication Module	2.3	3.5
3.5	API Layer	3.1, 3.2, 3.4	4.2, 5.1
4.1	NL Query Accuracy Testing	3.2	4.3
4.2	Integration & E2E Testing	3.1-3.5	4.3
4.3	Demo Rehearsal & UAT	4.1, 4.2	5.2
5.1	Infrastructure & CI/CD	3.3, 3.5	5.2, 6.1
5.2	Data Migration & Seeding	3.1, 5.1	6.2
6.1	Monitoring	5.1	6.2
6.2	Knowledge Transfer	All prior	6.3
6.3	Phase 2 Roadmap	Demo feedback	None

Risk Register

ID	Risk	Likelihood	Impact	Severity	Mitigation
R 1	LLM generates incorrect SQL leading to wrong financial answers in CEO demo	Medium	High	High	Technical spike (1.3) validates accuracy early. Build SQL validation layer. Pre-test all demo questions. Add confidence indicators to answers.
R 2	Spreadsheet format inconsistency across 50 files breaks ingestion pipeline	High	Medium	High	Data audit (1.2) identifies issues upfront. Build robust error handling and skip-on-failure logic. Prioritize cleanest files for demo.
R 3	Budget overrun due to scope expansion during development	Medium	Medium	Medium	Strict scope discipline: POC only, no production features. Weekly budget check. Cut scope before adding cost.

ID	Risk	Likelihood	Impact	Severity	Mitigation
R 4	OpenAI API latency causes slow query responses during demo	Low	Medium	Medium	Cache common queries. Pre-warm the API connection. Have fallback pre-computed answers for demo script questions.
R 5	CEO asks questions outside the supported domain, revealing system limitations	Medium	Low	Medium	Define supported query types. Add graceful fallback messages. Rehearse demo script to control question flow.
R 6	Data contains PII or sensitive financial information requiring compliance measures	Low	High	Medium	Data audit (1.2) checks for PII. If found, add data masking. For POC, use anonymized subset if needed.

Assumptions & Constraints

Assumptions

- The 50 spreadsheets are primarily in Excel (.xlsx) or CSV format with tabular structure (headers + rows).
- Sales and inventory data has a reasonably consistent schema across files (e.g., common columns like date, product, quantity, revenue).
- Weekly data updates are manual (user uploads new files). No real-time sync required.
- 5-10 users are internal team members. No public access or external authentication required.
- The CEO demo is the primary success criteria. Production readiness is out of scope for this phase.
- OpenAI GPT-4 API is available and within acceptable cost range for the query volume (~100-500 queries/week).
- The client has no existing infrastructure preferences or constraints (greenfield deployment).
- No regulatory compliance requirements (SOC 2, HIPAA, GDPR) apply to this POC phase.
- The development team has access to AI tooling (Claude Code) to accelerate development.

Constraints

- Budget is fixed at \$15K for the initial build. Scope will be cut before budget is exceeded.
- This is a POC/demo build. Production hardening, HA, auto-scaling, and advanced security are Phase 2.
- NL query accuracy target is 85%+ for common sales/inventory questions. Edge cases and complex multi-step queries may not be supported.
- The system is designed for 5-10 concurrent users. It will not scale beyond this without infrastructure changes.
- Data freshness is weekly. Sub-daily updates require pipeline rearchitecture (Phase 2).
- The NL query engine depends on the OpenAI API. Service outages or API changes are outside our control.